AI Engineer 포트폴리오
LLM 에이전트 · Python 백엔드

정은희

기술보다 그 기술을 쓰는 사람의 하루를 먼저 봅니다.
혼자 만들어 운영 중인 교환 서비스와 SKALA 팀 프로젝트를 중심으로 정리했습니다.

github.com/Eunhee-JJ
Google Scholar
2026년 11월

자기소개

기술보다 그 기술을 쓰는 사람의 하루를 먼저 봅니다.

XR 콘텐츠 제작 자동화 연구에 참여해 IEEE ISMAR-Adjunct 2024에 제1저자로 논문을 냈습니다. LLM이 물체의 쓰임새를 설명하면 그 결과를 Unity 장면에 적용하는 연구였는데, 가장 오래 붙잡고 있던 건 모델 자체보다 모델의 답을 프로그램이 읽을 수 있는 형태로 맞추는 일이었습니다. 그 작업이 제일 재미있었고, 그래서 AI를 서비스에 붙이는 개발 쪽으로 방향을 정했습니다.

지금은 **폴짝**이라는 서비스를 혼자 만들어 운영하고 있습니다. 저는 연극과 뮤지컬을 보며 배우 폴라로이드를 모으는데, 팬들끼리 폴라를 교환하려면 트위터에 목록을 올리고, 검색하고, DM으로 하나하나 조율해야 합니다. 직접 교환하면서 번거로웠던 부분을 줄이려고 만든 서비스입니다. 무엇을 만들지는 기술 목록이 아니라, 교환하는 사람이 어디서 시간을 쓰고 어디서 지치는지를 보고 정합니다.

AI를 붙인 것도 같은 이유였습니다. 사용자가 트위터에 이미 쓴 교환글을 폴짝에 두 번 쓰지 않도록, 캡처를 읽어 등록 화면을 채워 주는 기능을 만들었습니다. 처음에는 글자 인식 결과를 전부 LLM에 넘겼는데, 운영비를 아끼려고 고른 2코어 서버에서는 한 장에 1분 넘게 기다려도 결과가 나오지 않았습니다. 더 작은 모델도 같은 결과였습니다. 실패한 캡처를 다시 보니 대부분 정해진 형식으로 쓰여 있어서, 형식대로 쓰인 글은 규칙으로 먼저 읽고 나머지만 LLM에 보내도록 바꿨습니다. 같은 캡처 기준으로 66초 걸리던 응답이 6초가 됐습니다. 캡처를 올리고 기다리는 사람에게는 더 큰 모델보다 빨리 오는 답이 중요하다고 판단했습니다.

SK AI 교육과정 SKALA에서는 팀 프로젝트로 멀티 에이전트 보고서 생성, 에이전트용 YouTube 검색 도구, 모델 재학습 파이프라인을 맡았습니다. 제 코드를 받아 쓰는 팀원도 사용자라고 생각해서, 시작 전에 입력과 출력 형식, 실패했을 때 돌려줄 값을 먼저 정해 두는 습관이 생겼습니다.

지금까지는 서버 한 대, 작은 규모에서 고민해 왔습니다. 다음에는 에이전트를 쓰는 사람이 훨씬 많은 환경에서, 그 사람들이 기다리지 않고 믿고 쓸 수 있는 백엔드를 만들고 싶습니다.

주로 쓰는 기술

Python, FastAPI, Java, Spring Boot, LangChain, LangGraph, Spring AI, PostgreSQL(pgvector), Docker, GitHub Actions, Cloud Run, MLflow, PyTorch, scikit-learn, LightGBM

목차

	프로젝트	구분	요약	쪽
1	**폴짝: 폴라로이드 교환 서비스**	개인	팬으로서 불편했던 교환을 서비스로, 캡처 등록 66초 → 6초	4-5
2	**BDI 예측 모델 재학습**	팀	평가를 통과한 모델만 운영에 반영	6
3	**기술 평가 보고서 멀티 에이전트 (팀)**	팀	근거가 없는 판단은 보고서에 넣지 않음	7
4	**기술 평가 보고서 멀티 에이전트 (개인 개선)**	개인	고정 순서를 근거, 평가, 예산을 보고 판단하는 실행으로	8
5	**마케팅 리서치 에이전트의 YouTube 도구**	팀	실제 API에서만 나던 실패를 찾아 수정	9
6	**티켓 환불 상담 에이전트**	개인	조회는 본인 것만, 환불은 사람이 승인	10
7	**신약 후보물질 독성 예측**	개인	단계별 튜닝, 통계로 확인한 개선만 채택	11
	기타 경험		SKALA, Kubernetes 실습, LoRA, XR 연구, SafePass	12

폴짝: 폴라로이드 교환 서비스

기간	2026.08 - 운영 중
형태	개인 프로젝트 (기획, 개발, 운영 1인)
기술	Spring Boot, Vue, PostgreSQL, GitHub Actions, Cloud Run

개요

폴짝은 연극·뮤지컬 팬들이 배우 폴라로이드를 교환할 상대를 찾고, 합의까지 할 수 있게 돕는 웹 서비스입니다. 기획부터 개발, 배포, 운영까지 혼자 맡고 있습니다. 이 쪽은 서비스 자체를, 다음 쪽은 캡처를 읽어 등록을 채워 주는 AI 기능을 다룹니다.

배경

배우 폴라로이드는 공연 이벤트에서 한 장씩만 받을 수 있어서, 원하는 폴라를 얻으려면 팬끼리 교환해야 합니다. 지금은 트위터에 가진 것과 원하는 것을 올리고, 검색으로 상대를 찾고, DM으로 조건을 맞춥니다. 저도 이렇게 교환해 왔고, 검색 결과를 끝없이 넘기는 일과 매번 정중한 문구를 쓰는 일이 가장 번거로웠습니다.

문제

혼자 화면과 서버를 함께 만들다 보니 한쪽을 고치면 다른 쪽이 어긋나는 일이 잦았습니다. 장터 목록 한 페이지를 그리는 데 DB 쿼리가 47번 나가고 있었고, 여러 사람이 같은 폴라에 동시에 제안하면 한 장이 두 교환에 묶일 수 있었습니다.

한 일

API 명세를 먼저 쓰고 화면 쪽 타입, 목업 서버, 백엔드가 모두 이 명세를 따르게 했습니다. 현재 59개 경로를 파일 하나로 관리합니다.

목록은 화면에 보이는 한 페이지만 읽고 이름, 상태, 개수를 한 번에 가져오도록 바꿨습니다. 교환은 두 사람이 모두 동의한 시점에만 폴라를 잠그고, 제안을 만들기 직전에 해당 데이터를 잠가 상태를 다시 확인합니다.

결과

장터 한 페이지의 쿼리 수가 47회에서 7회로 줄었습니다(개발용 데이터 기준). PR마다 테스트가 자동으로 돌고, main에 합치면 Cloud Run 배포까지 이어집니다.

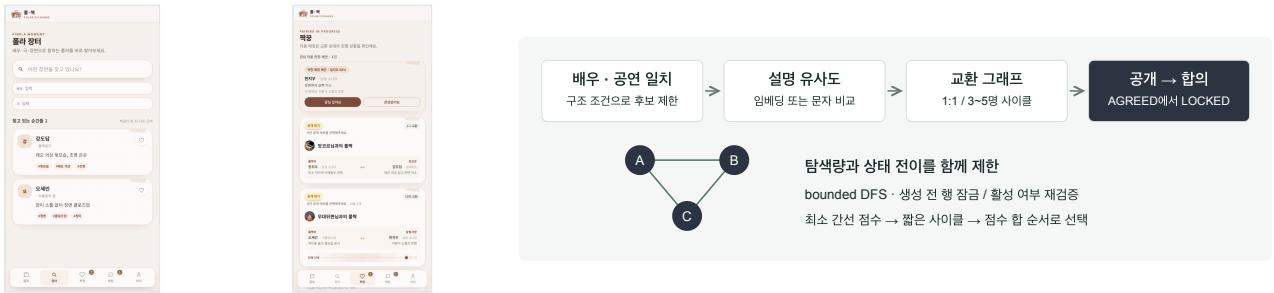


그림 1. 왼쪽: 장터와 교환 제안 화면. 오른쪽: 교환이 성사되기까지의 상태 흐름.

폴짝: 캡처 자동 등록 (이어서)

기간	2026.09 - 10
형태	폴짝의 별도 AI 서버 (커밋 12건 모두 직접 작성)
기술	Python, FastAPI, PaddleOCR, Qwen3-4B (llama.cpp), Docker, Cloud Run

배경

폴짝으로 옮겨 오는 사용자는 이미 트위터에 교환글을 올려 둔 상태입니다. 그 내용을 처음부터 다시 입력하게 하면 시작부터 지칩니다. 캡처만 올리면 배우, 작품, 설명이 채워지게 하고 싶어서, 글자 인식과 LLM을 본 서버와 분리한 AI 서버를 만들었습니다.

문제

처음에는 글자 인식 결과를 LLM에 넘겨 항목을 뽑았습니다. 운영비를 아끼려 고른 2 vCPU, 메모리 4GB 환경에서 같은 캡처로 네 번 시험했더니 LLM이 매번 60초 가까이 쓰고도 실패했고, 결국 응답은 예비로 둔 규칙 결과였습니다. 더 작은 모델(Qwen2.5-1.5B)로도 해 봤지만 시간 초과로 끝났습니다.

한 일

실패한 캡처를 다시 보니 대부분 “배우: 작품: 설명”처럼 정해진 형식이었습니다. 그래서 규칙으로 먼저 읽고, 규칙이 항목을 하나도 찾지 못한 글만 LLM에 보내도록 순서를 바꿨습니다. LLM은 한 번에 한 건만 처리하고, 이미 처리 중이면 요청을 쌓아 두지 않고 바로 실패를 알려 사용자가 직접 입력으로 넘어가게 했습니다.

본 서버가 다음 행동을 정할 수 있게 “읽을 내용이 없음”과 “모델 준비 중”을 서로 다른 응답으로 나눴고, 모델 로딩 상태는 따로 조회할 수 있게 했습니다.

결과

같은 캡처의 응답 시간이 66초에서 6초로 줄었고, 15개 항목 중 하나도 못 읽던 캡처에서 13개를 읽었습니다. 규칙이 일부만 찾은 항목은 사용자가 화면에서 고쳐 등록합니다.

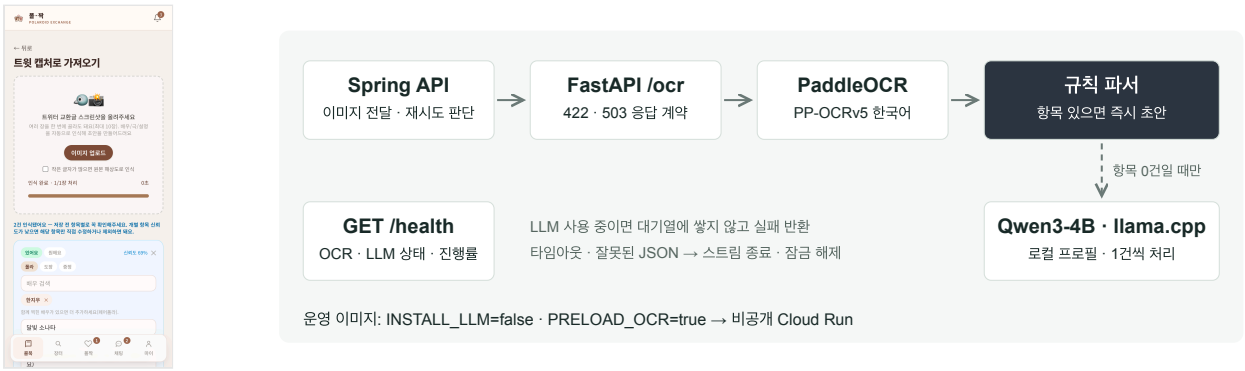


그림 2. 왼쪽: 캡처 등록 화면(목업 데이터). 오른쪽: 캡처 처리 순서, 규칙으로 항목을 찾으면 LLM을 부르지 않습니다.

시간은 같은 이미지를 순서대로 처리해 측정했습니다. 13/15는 사용자 확인을 거친 잠정 정답 기준입니다.

BDI 예측 모델 재학습

기간	2026.10
형태	SKALA 팀 프로젝트 (재학습, 검증, 버전 관리 담당)
기술	Python, MLflow, Docker

개요

발틱 건화물 운임지수(BDI)의 다음 날 변동을 예측해 API로 제공하는 팀 프로젝트입니다. 예측 모델은 팀이 만들었고, 저는 서비스 중인 모델을 다시 학습하고 교체하는 부분을 맡았습니다.

배경

운임지수는 시장 상황에 따라 흐름이 바뀌어서, 한 번 만든 모델을 그대로 쓰면 예측이 점점 맞지 않게 됩니다. 그래서 새 데이터로 다시 학습하고, 더 나은 모델로 갈아 끼우는 과정이 필요했습니다.

문제

재학습을 API 요청 안에서 실행하면 학습이 끝날 때까지 응답이 멈추고, 요청이 겹치면 학습이 두 번 돌았습니다. 더 큰 문제는 새로 학습한 모델이 이전보다 나빠도 그대로 운영에 올라간다는 점이었습니다.

한 일

재학습은 백그라운드에서 돌리고, 동시에 하나만 실행되게 막았습니다. 새 모델은 최근 21거래일 데이터로 평가해서, 오늘 값이 내일도 그대로라고 보는 단순 기준보다 오차가 작을 때만 운영 버전으로 바꿉니다. 통과 여부와 이유는 MLflow에 함께 기록합니다.

결과

기준을 넘지 못한 모델은 기록만 남고 기존 모델이 계속 서비스됩니다. 지금 어떤 버전이 왜 운영 중인지 API와 로그로 확인할 수 있습니다.

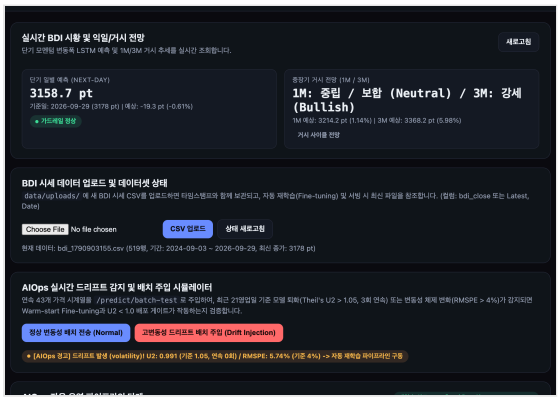




그림 3. 왼쪽: 예측 대시보드. 오른쪽: 재학습에서 운영 버전 교체까지의 흐름.

기술 평가 보고서 멀티 에이전트

모든 판단에 근거가 붙는 보고서

기간	2026.09
형태	SKALA 6인 팀 프로젝트 (보고서 작성, 품질 검사, 다음 단계 결정 담당)
기술	Python, LangGraph, Jinja2

개요

KV Cache 관련 기술이 얼마나 실용화에 가까운지 평가하는 보고서를 여러 에이전트가 나눠 쓰는 6인 팀 프로젝트입니다. 저는 조사 결과로 보고서를 쓰고 검사하는 단계와, 다음에 어느 에이전트를 실행할지 정하는 부분을 맡았습니다.

배경

기술 평가 보고서는 판단마다 근거가 있어야 믿을 수 있습니다. 논문, 시장, 이해관계자 조사는 다른 팀원이 맡았고, 제 몫은 이 조사 결과를 근거가 드러나는 보고서로 묶는 일이었습니다.

문제

처음 구조는 정해진 순서대로 실행돼서, 근거가 부족한 관점이 있어도 그대로 보고서까지 넘어갔습니다. 또 LLM에게 보고서 전체를 다듬게 하면 근거에 없는 문장이 섞일 수 있었습니다.

한 일

관리 역할의 에이전트가 지금까지의 검증 결과와 재시도 횟수를 보고 다음 단계를 고르도록 바꿨습니다. 보고서의 틀과 인용 번호는 템플릿이 채우고, LLM은 근거 문장을 자연스럽게 옮기는 데만 씁니다. 근거가 없는 항목은 “판단 불가”로 남기고, 검사에서 부족한 관점이 나오면 그 조사만 다시 실행합니다.

결과

보고서의 모든 판단에 인용 번호가 붙어 참고문헌까지 따라갈 수 있습니다.

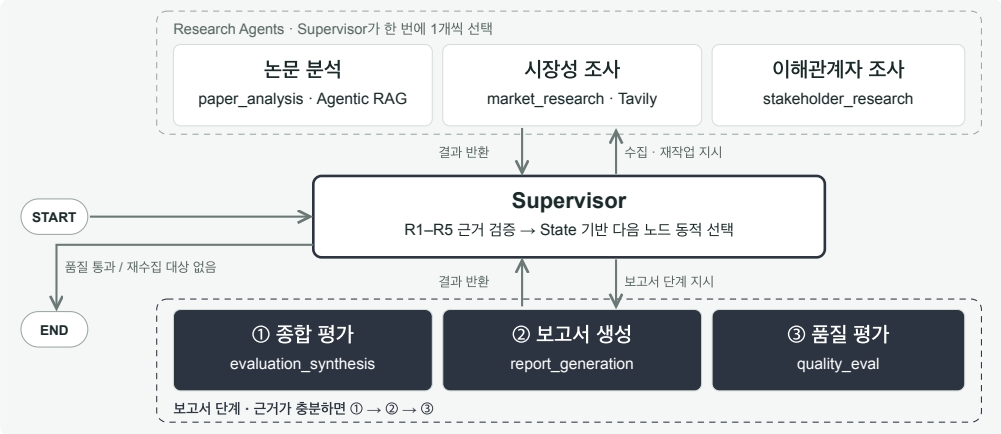


그림 4. 에이전트 구성. 진하게 표시한 세 단계를 맡았습니다.

기술 평가 보고서 멀티 에이전트

근거, 평가, 예산을 보고 스스로 판단하는 에이전트

기간	2026.10
형태	개인 (기존 에이전트 코드 개선)
기술	Python, LangGraph, FastAPI, tiktoken

개요

여러 에이전트가 자료를 모아 기업별 기술 수준을 비교하는 보고서를 씁니다. 늘 같은 순서로만 돌던 기존 에이전트를, 상황을 보고 다음 단계를 스스로 정하도록 바꿨습니다.

문제

기존 코드는 조사, 기업 분석, 보고서 작성을 늘 같은 순서로 실행했습니다. 보고서 검사는 300자가 넘는지만 봐서 근거 없는 보고서도 통과했고, 한 번 실행에 드는 비용에도 제한이 없었습니다.

한 일

상황을 보고 다음 단계 선택. 질문 유형을 먼저 나누고, Supervisor가 근거가 충분한지, 검사를 통과했는지, 돈이 남았는지를 보고 재검색, 기업별 분석, 재작성 중 다음 일을 고릅니다. 기업별 분석은 비교 질문일 때만 동시에 돌립니다.

단계마다 수치로 검사. 근거 수집 후, 기업 분석 후, 보고서 작성 후에 검사를 둡니다. 최종 보고서는 LLM 평가자가 근거와 맞는 주장 비율(0.8 이상), 근거 번호가 없는 주장 비율(0.1 이하) 등 5개 항목으로 채점합니다. 떨어진 이유는 다음 시도에 그대로 전달하고, 재시도는 최대 두 번입니다.

예산 안에서 끝까지. LLM을 부르기 전에 예상 비용을 잔액에서 먼저 떼어 두고 끝나면 정산해서, 동시에 도는 분석도 한도를 넘지 못합니다. 보고서 작성과 검사 몫이 남지 않으면 재검색과 재분석을 건너뛰고, 그마저 없으면 LLM 없이 요약본으로 마칩니다.

결과

예산이 빠듯한 실행에서 Supervisor는 근거 검사를 통과한 뒤 잔액이 모자란 기업별 분석을 건너뛰었고, 최종 평가에서 3개 항목이 떨어졌지만 재작성 대신 한도 안에서 보고서를 마쳤습니다. 판단마다 이유가 화면에 남습니다.

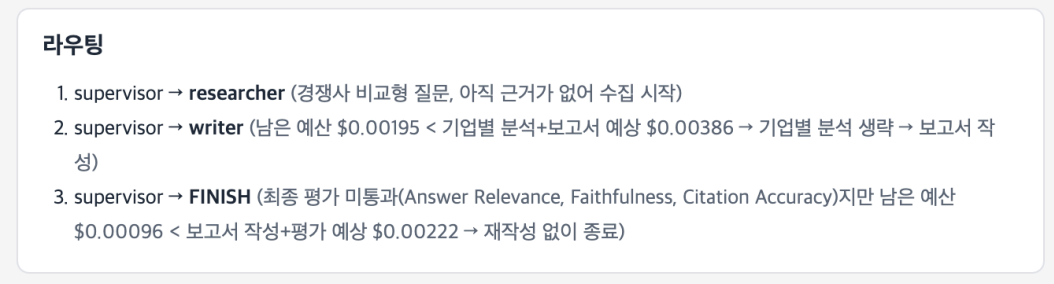


그림 5. 판단 기록(한도를 0.002달러로 낮춰 시험). 근거 수집 → 예산 부족으로 기업별 분석 생략 → 재작성 없이 종료.

마케팅 리서치 에이전트의 YouTube 도구

기간	2026.09
형태	SKALA 팀 프로젝트 (YouTube 검색, 수집 도구 담당)
기술	Python, LangChain, YouTube Data API

개요

마케터의 경쟁사 분석을 돕는 에이전트를 만든 팀 프로젝트입니다. 저는 에이전트가 호출하는 YouTube 검색, 수집 도구와 경쟁사 분석 흐름을 만들었습니다.

배경

경쟁사 분석 초안을 쓰려면 경쟁사 채널을 찾고, 영상과 통계를 기간별로 모아 정리해야 해서, 마케팅 인턴 경험이 있는 팀원 기준으로 손으로 30분쯤 걸리는 일이었습니다. 에이전트가 이 일을 대신하려면 YouTube를 정확히 조회하는 도구가 먼저 있어야 했습니다.

문제

가짜 데이터로 만든 테스트는 모두 통과했는데, 실제 YouTube에 연결해 처음부터 끝까지 돌려 보니 채널을 찾지 못했습니다. 실제 응답 구조가 테스트용 데이터와 달랐습니다.

한 일

실제 응답을 확인하고, 같은 실패가 나는 테스트를 먼저 추가한 다음 고쳤습니다. 고친 뒤에는 기존 테스트를 다시 돌려 다른 기능이 깨지지 않았는지 확인했습니다. 도구마다 받을 값을 정해 두어 잘못된 호출은 API를 부르기 전에 걸렀고, YouTube 쪽이 실패해도 에이전트 전체가 멈추지 않게 했습니다.

결과

팀원이 손으로 30분쯤 들이던 경쟁사 분석 초안이 7초 안에 나옵니다.

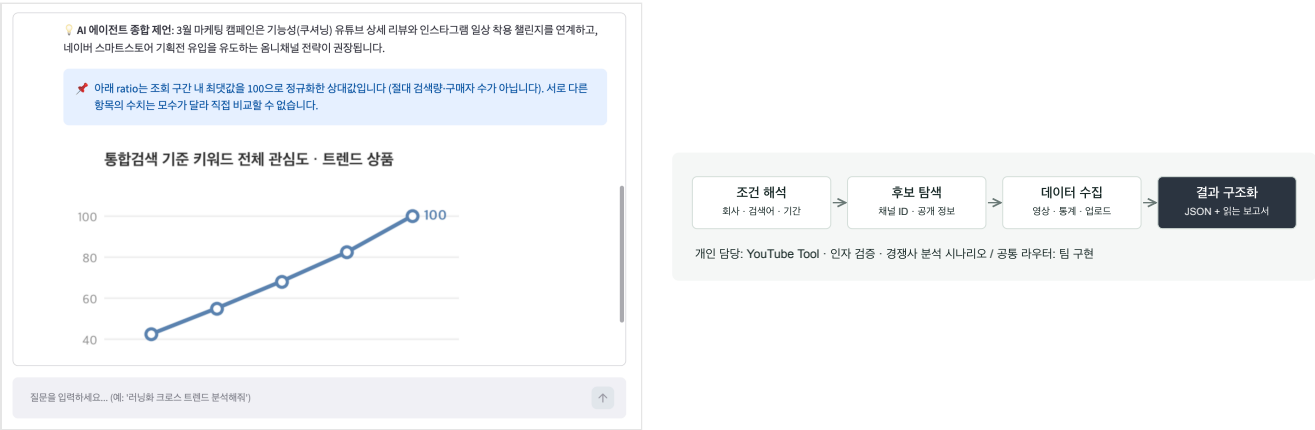


그림 6. 왼쪽: 경쟁사 분석 결과 화면. 오른쪽: 에이전트가 YouTube 도구를 부르는 흐름.

티켓 환불 상담 에이전트

기간	2026.08
형태	개인 프로젝트
기술	Java, Spring Boot, Spring AI, PostgreSQL(pgvector)

개요

공연 티켓 환불 문의에 답하는 상담 에이전트를 혼자 만든 프로젝트입니다. 규정 검색, 예매 조회, 담당자 승인 절차를 하나의 상담 흐름으로 연결했습니다.

배경

환불이 되는지는 규정과 그 사람의 예매 내역을 같이 봐야 알 수 있습니다. 답이 틀리면 고객이 손해를 보거나 잘못된 환불이 나갈 수 있어서, 편리함보다 정확함이 먼저인 상담이었습니다.

문제

LLM에게 그대로 맡기면 규정을 짐작해서 답하거나, 다른 사람의 예매를 조회하거나, 환불을 직접 처리해 버릴 수 있었습니다.

한 일

규정은 문서에서 찾아 인용하게 하고, 예매 조회와 환불 금액 계산은 정해진 기능(Tool)으로 분리했습니다. 이 기능은 로그인한 사람의 ID로만 조회하게 해서 다른 사람의 예매는 있는지조차 알 수 없습니다. 환불처럼 상태를 바꾸는 요청은 실행하지 않고 담당자 승인 대기로 넘깁니다.

결과

출처, 권한, 행동을 하나씩 채점하는 평가 시나리오 6개를 모두 통과했습니다.

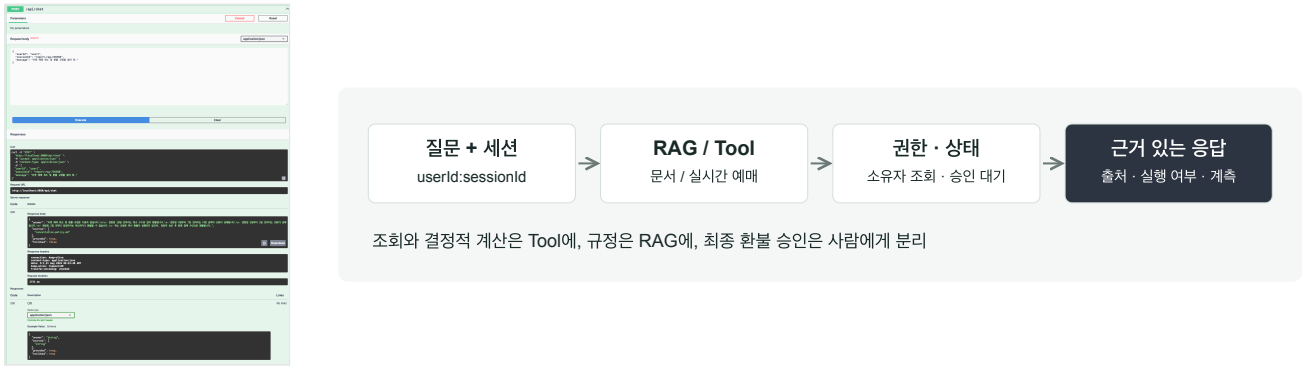


그림 7. 왼쪽: 규정 인용 답변 화면. 오른쪽: 상담 처리 흐름.

신약 후보물질 독성 예측

기간	2026.10
형태	SKALA 개인 과제 (실험 설계, 구현, 제출 1인, PR 13건)
기술	Python, LightGBM, Optuna, scikit-learn, RDKit

개요

분자 구조로 화합물의 독성 여부를 분류하는 SKALA 개인 과제입니다. 기준 모델에서 출발해 피처, 모델, 하이퍼파라미터, 앙상블 순서로 한 단계씩 바꾸며 채점 기준인 무독성 F1을 올렸고, 모든 단계를 같은 분할과 지표로 비교했습니다.

문제

처음 쓴 채택 규칙은 점수 상승폭이 그 실행의 표준편차보다 크면 개선으로 봤습니다. 이 규칙은 표준편차가 우연히 작게 나온 실행에 유리했습니다. 실제로 RDKit 피처를 더한 실험이 채택됐는데, 그 상승폭은 모델을 LightGBM으로 바꾼 효과와 똑같았습니다. 반대로 LightGBM 교체 자체는 탈락했습니다.

한 일

판정 기준을 같은 데이터 조각(폴드)끼리 짝지어 비교하는 통계 검정으로 바꿨습니다. 다시 판정하자 RDKit 피처는 효과가 없었고($p = 0.50$), LightGBM 교체는 채택됐습니다($p = 0.049$).

LightGBM은 Optuna로 40회 탐색했습니다. 탐색에 쓰는 폴드는 평가 폴드와 다르게 섞었는데, 탐색 중 최고 점수 0.8257이 평가 폴드에서는 0.8213이었습니다. 같은 폴드를 썼다면 0.004만큼 부풀려진 점수를 믿었을 것입니다. 튜닝으로 교차 검증 F1이 0.8141에서 0.8213으로 올랐습니다($p = 0.030$).

튜닝한 모델에 다른 모델을 섞은 앙상블 7가지도 시험했지만 모두 단독보다 낮아 채택하지 않았습니다. 분류 기준값(임계값)은 학습 데이터 안에서만 골랐습니다.

결과

교차 검증 F1이 0.7929에서 0.8213으로 올랐습니다. 테스트셋은 마지막에 한 번만 평가했고 F1 0.8266으로 검증셋(0.8210)과 비슷해 과적합 징후는 없었습니다. 찾은 하이퍼파라미터 여러 개가 탐색 범위 끝에 붙어 있어, 범위를 넓히면 더 오를 여지가 있습니다. 무작위 분할 기준이라 구조가 많이 다른 분자에서의 성능은 확인하지 않았습니다.

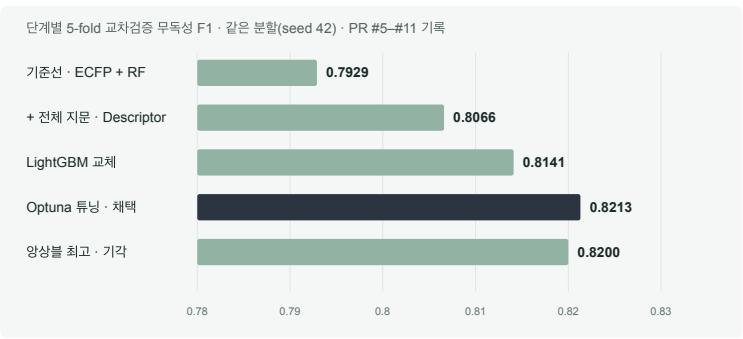


그림 8. 단계별 교차 검증 무독성 F1. 튜닝은 채택, 앙상블은 기각

기타 경험

SK AI 전문가 양성과정 SKALA

2026.07 - 2026.12, 참여 중

Java, Spring Boot, Vue, Docker, MSA, RAG, 파인튜닝, 에이전트 개발을 배우고 있습니다. 앞의 팀 프로젝트 세 개와 개인 과제 두 개가 이 과정에서 나왔습니다.

Kubernetes 배포 실습

SKALA 강의 실습

Spring Boot 앱을 이미지로 만들어 Kubernetes에 올리고, 상태 확인(liveness/readiness)을 연결했습니다. 개발과 운영 환경은 Kustomize로 설정과 인스턴스 수(개발 1개, 운영 3개)를 나눠 관리했습니다.

사내 규정 질의응답 모델 파인튜닝

SKALA 팀 실습, 학습 조건 설계와 비교 담당

Qwen2.5-1.5B를 LoRA로 학습해 원래 모델과 같은 질문 20개로 비교했고, 20개 모두 학습한 모델의 답이 낫다는 판정을 받았습니다. 팀이 시험한 18개 조건 중 점수가 가장 높은 조건이었습니다.

XR 상호작용 저작 자동화 연구

IEEE ISMAR-Adjunct 2024, 제1저자

LLM으로 물체의 쓰임새에서 상호작용 방식과 위치를 뽑아 Unity 장면에 연결하는 연구입니다. 자기소개에 쓴 진로 방향이 여기서 정해졌습니다.

SafePass

팀 프로젝트, 스크럼 마스터

여러 서비스로 나뉜 팀 프로젝트에서 칸반과 데일리 스탠드업을 운영하며 서비스 간 API 의존 관계와 막힌 작업을 조율했습니다. 작업 등록, 적합성 확인, 교육 배정으로 이어지는 MVP를 기간 안에 완성했습니다.

github.com/Eunhee-JJ